

Great, you're back. So, as you can see, these are just some properties that we're now setting the values for. The 'id' attribute contains a string that tells the add-on suite that you're developing the add-on for Firefox, as opposed to ThunderBird, or another Mozilla product. Also, rest easy in the knowledge that it's provided by default by the builder when you run `cfx init`.

The rest of the attributes are more easily understandable, with the first two declaring the 'name' and 'fullName' values for the add-on. We then proceed to tell the builder the package dependencies to compile for our add-on. Don't worry about the contents of that array – we'll get to it later. Then, we'll provide a small description of the add-on that we're building. For now, just make a mental note of what's going on. All of that is then followed by the author's name, the license under which the code is to be released and the current development version. In order to have the add-on working, it should be allowed to be executed in the 'private browsing' mode as well.

The version number is decided by semantic versioning. To explain this simply, it's done using the `x.y.z` structure. Here, `x` represents the instance when you make absolutely incompatible changes from your previous version, and/or have created an experience significantly different (hopefully for the better) from the last one. Now, `y`, one the other hand, is for the minor revisions that are made over the due course of a release. Lastly, `z` stands for the patches that you might need to release in order to fix bugs that inevitably land in any piece of software that is more than 100 lines in extent.

Now, let's take a look at possibly the single largest file that we'll encounter in the course of this exercise. The `chrome.manifest` file, which is about...one line in its entirety. Hence, gotcha!

- `<code>`
- 
- `content fx-statistics data/`
- 
- `</code>`

Let's examine what this behemoth is trying to accomplish. Now, add-ons work on a registry basis, which means that they need to be acknowledged by the underlying engine for an action to take effect. After this acknowledgement is given, an add-on can be made available for access (in addition to whatever other mode you've chosen, because it's generally bad practice to direct the user to a dedicated URL for using your extension). This is where